

Machine Learning-Based Stress Level Prediction using Random Forest

NIDHI BISEN | nidhibisen242005@gmail.com | 0801IT243D05

Abstract

Introduction: In recent years, students and young professionals face increasing stress due to academic workload, social pressures, and lifestyle challenges. Accurate prediction of stress levels is essential for early intervention and effective mental health management.

Methods: In this study, a stress level prediction system was developed using machine learning techniques on the StressLevelDataset. Multiple algorithms, including Logistic Regression, SVM, KNN, Decision Tree, XGBoost, and Random Forest, were implemented and compared to identify the optimal model. Hyperparameter tuning was applied to improve model performance. Additionally, evaluation metrics such as accuracy, precision, recall, F1-score, and ROC-AUC were used to validate the model.

Results: Among all models, the optimized Random Forest classifier achieved the highest accuracy of **89.09%**, outperforming other models. The model demonstrated balanced performance across all stress categories, with strong precision and recall values. Feature importance analysis indicated that factors such as sleep quality, anxiety, and academic pressure significantly influence stress levels.

Discussion: The results highlight the effectiveness of ensemble learning methods in stress prediction tasks. The developed system was further deployed using a Flask-based web application, enabling real-time stress level prediction. This approach provides a practical solution for early stress detection and can support mental health awareness and intervention strategies.

Keywords: Stress Level Prediction, Random Forest, Machine Learning, Ensemble Learning, Classification, Student Stress Analysis, Feature Importance, Flask Web Application, Data Preprocessing, Multi-class Classification

1. Introduction

In today's society, students and young adults face increasing stress due to academic pressure, social challenges, and career expectations. These factors significantly impact mental health, often leading to anxiety and reduced well-being. Early and accurate stress detection is therefore essential for effective intervention. However, traditional assessment methods are subjective and insufficient for capturing complex stress patterns.

Recent advancements in machine learning have enabled data-driven approaches for stress prediction. Various classification algorithms such as Logistic Regression, SVM, KNN, Decision Tree, XGBoost, and Random Forest have been widely applied. Among these, ensemble methods like Random Forest are particularly effective in handling non-linear relationships, reducing overfitting, and improving prediction accuracy.

Key Contributions of This Work

A comparative analysis of multiple machine learning algorithms was conducted, where the optimized Random Forest model achieved the best performance with an accuracy of 89.09%, outperforming Logistic Regression, SVM, XGBoost, Decision Tree, and KNN.

- Model evaluation using metrics such as precision, recall, F1-score, and ROC-AUC confirmed balanced and reliable performance across all stress categories.
- Feature importance analysis identified key factors such as sleep quality, anxiety, and academic pressure as significant contributors to stress levels.
- A Flask-based web application was developed for real-time stress prediction, enhancing the practical applicability of the system.

This research provides a data-driven and scalable approach for stress detection, offering valuable insights for educators, students, and mental health professionals to take timely and effective intervention measures.

Dataset Overview

Table 1 :- Dataset information

Attribute	Detail
Dataset	StressLevelDataset (Kaggle)
Samples	1,100 instances (approximately)
Features	~20–21 structured numerical features (psychological, physiological, academic, social)
Task	Multi-class Classification — Low Stress (0), Medium Stress (1), High Stress (2)
Models Used	Logistic Regression, SVM, KNN, Decision Tree, XGBoost, Random Forest (Baseline & Optimized)
Best Model	Random Forest (Optimized) — Accuracy: 89.09% on 220-sample test set

2. Literature Review

Recent advancements in affective computing have highlighted the importance of physiological signals in stress detection, particularly tissue oxygen saturation (StO₂) derived from hyperspectral imaging (HSI). StO₂ serves as a non-contact physiological indicator capable of reflecting emotional states effectively. However, the high dimensionality of hyperspectral data limits real-time applications. To address this, Linear Prediction (LP) was used to reduce 106 spectral bands to 8 significant bands while maintaining performance. The system achieved an accuracy of 84.44% using a Bayes classifier, demonstrating that dimensionality reduction can enhance efficiency without compromising accuracy [1].

Stress prediction systems have also been developed as user-centric applications for monitoring and managing stress. These systems utilize machine learning algorithms such as neural networks and decision trees to analyze user inputs and predict stress levels. Such applications not only identify stress levels but also suggest appropriate measures for stress management. Their ability to be integrated into web and mobile platforms makes them scalable and practical solutions for real-world deployment [2].In engineering domains, stress prediction has been widely studied for material reliability and failure analysis. A lifetime prediction model for stress-induced voiding (SIV) in copper interconnects uses the Gompertz function to model stress behavior and estimate material lifespan [3].

Similarly, research on solder connections highlights the significance of stress triaxiality in predicting ductile fracture and failure mechanisms. These studies emphasize the role of computational modeling in predicting structural failures and improving system design [4].In biomedical engineering, mechanical stress plays a

critical role in tissue development and cell proliferation. Researchers developed 3D models to analyze shear stress distribution in porous scaffolds using computational fluid dynamics (CFD). The findings indicate that shear stress is linearly proportional to fluid velocity and significantly influenced by scaffold porosity. These predictive models are useful in optimizing tissue engineering processes and improving biological outcomes [5].

Thermal stress analysis in microelectronics packaging is another important area of research. Analytical models have been proposed to evaluate interfacial stress caused by mismatched coefficients of thermal expansion (CTE). These models consider real-world imperfections such as voids in adhesive layers and provide more accurate predictions compared to conventional methods. Such findings are essential for improving the reliability and durability of microelectronic systems [6].

Financial stress among students has become a major concern affecting academic performance and retention rates. Machine learning frameworks incorporating Decision Trees, K-Nearest Neighbors (KNN), Artificial Neural Networks (ANN), and ensemble methods like Voting Classifiers have been widely applied. Among these, LightGBM demonstrated superior performance with an accuracy of 91.16%. Advanced preprocessing techniques such as ADASYN and MinMax scaling further enhanced prediction accuracy, enabling early identification of at-risk students [7].

Mental stress prediction among students has also been extensively explored using advanced machine learning and deep learning techniques. Hybrid models combining Support Vector Machines (SVM) with Dolphin optimization achieved an accuracy of 99.8% and an F1-score of 0.99, indicating highly reliable performance [8]. Similarly, deep learning approaches such as TriBranch CNN and Fully Conditional Temporal Bayesian Networks (CTBN) have outperformed traditional models, achieving accuracy levels above 93% [10].

Traditional machine learning algorithms have also been widely used for stress prediction. Models such as Random Forest, Logistic Regression, and KNN have demonstrated strong performance, with Random Forest achieving accuracy up to 97.45% and KNN achieving approximately 88% accuracy in student datasets. These findings indicate that both ensemble and instance-based learning methods are effective in capturing complex stress-related patterns [9], [11].

Recent research has also explored advanced techniques such as clustering and swarm intelligence to improve prediction accuracy. Possibilistic Fuzzy C-Means (PPFCM) combined with Chameleon Swarm Algorithms enables effective segmentation and classification of stress levels, particularly in high-stress environments such as healthcare. Additionally, improved Apriori algorithms have been used for association rule mining to identify relationships between psychological factors, providing interpretable insights into stress prediction [13], [14].

Overall, the literature indicates that stress prediction is a multidisciplinary field integrating machine learning, biomedical engineering, and material science. Although significant progress has been made in improving prediction accuracy and efficiency, challenges such as data quality, generalization, and real-time implementation remain. Future research should focus on developing scalable, accurate, and user-friendly systems for early stress detection and effective management [16].

3. Methodology

3.1 System Architecture

3.1.1 Model A — Logistic Regression (Baseline Linear Model)

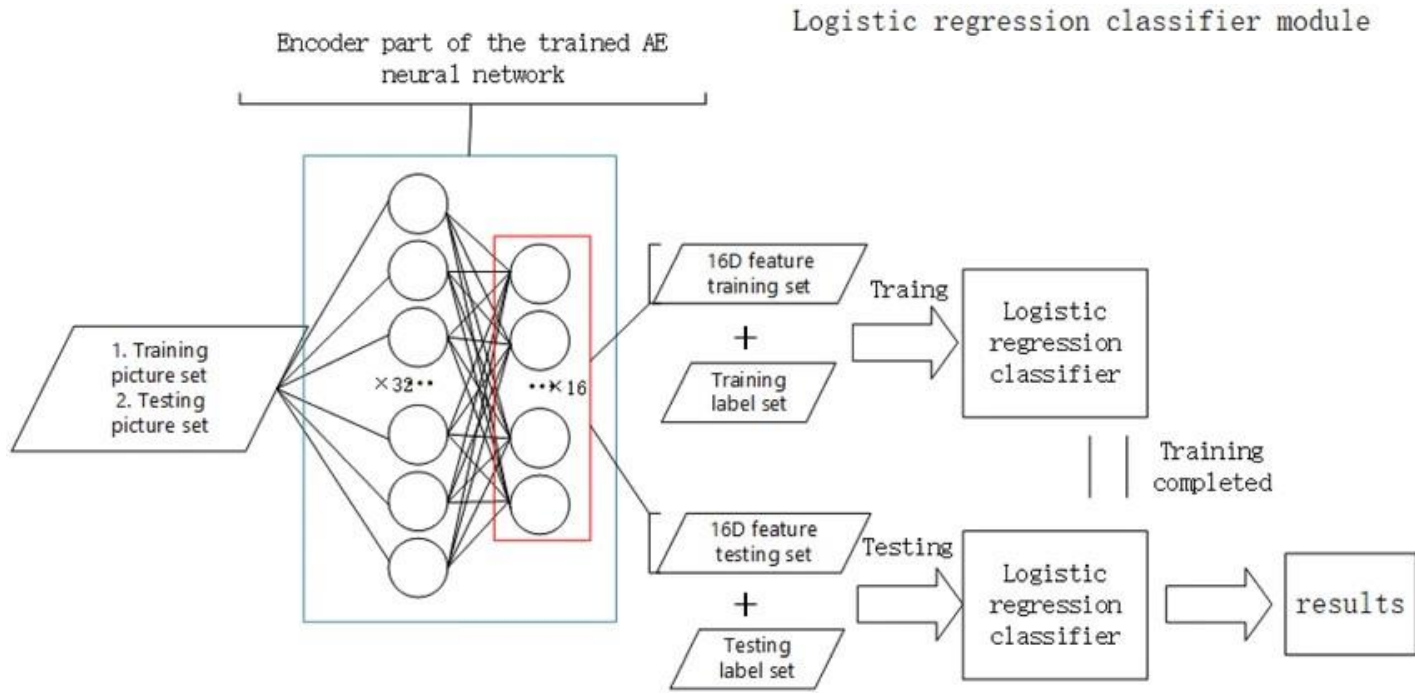


Figure 1: Architecture of the Logistic Regression-Based Stress Classification Model

The figure 1 shows that the model takes structured numerical input features such as anxiety level, depression, sleep quality, academic performance, and social factors. These input features are first preprocessed and normalized using techniques like StandardScaler to ensure uniform scaling. The processed features are then passed to the Logistic Regression model, which computes a linear combination of the inputs and applies a sigmoid (or softmax for multi-class) function to estimate class probabilities. The model learns weights for each feature to determine their influence on stress levels. Finally, the class with the highest probability is selected as the predicted stress category (low, medium, or high).

3.1.2 Model B — Support Vector Machine (SVM) Classification Model

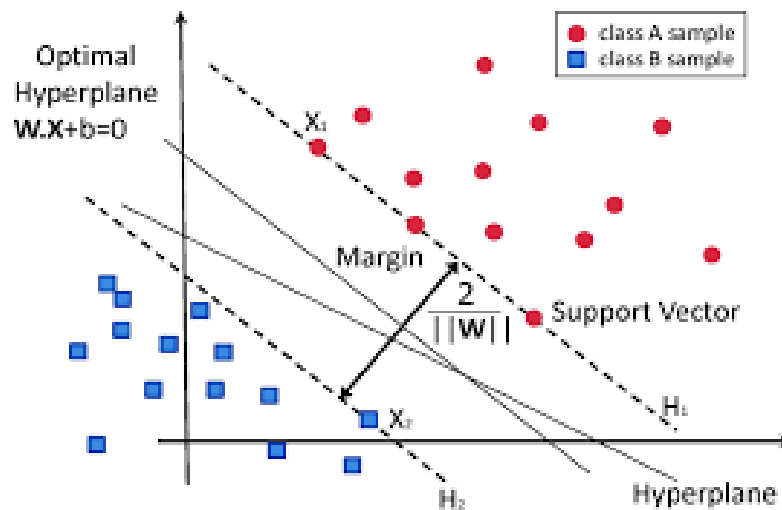


Figure 2: Architecture of the SVM-Based Stress Prediction Model

The figure 2 shows that the model takes preprocessed numerical features as input and maps them into a higher-dimensional feature space using kernel functions. The Support Vector Machine (SVM) then identifies an optimal hyperplane that separates different stress level classes with maximum margin. Support vectors, which are the most critical data points, define this boundary. The model is effective in handling non-linear relationships between features. After training, the SVM predicts the class of new input data based on its position relative to the decision boundary. The final output corresponds to the predicted stress level category. The Support Vector Machine (SVM) is used as a **robust classification model** that can handle both linear and non-linear stress patterns.

The support vectors play a crucial role in defining the decision boundary. If the stress data is not linearly separable, kernel functions (like RBF or polynomial) can be applied to transform it into a higher-dimensional space.

3.1.3 Model C — Ensemble Model (Random Forest Optimized)

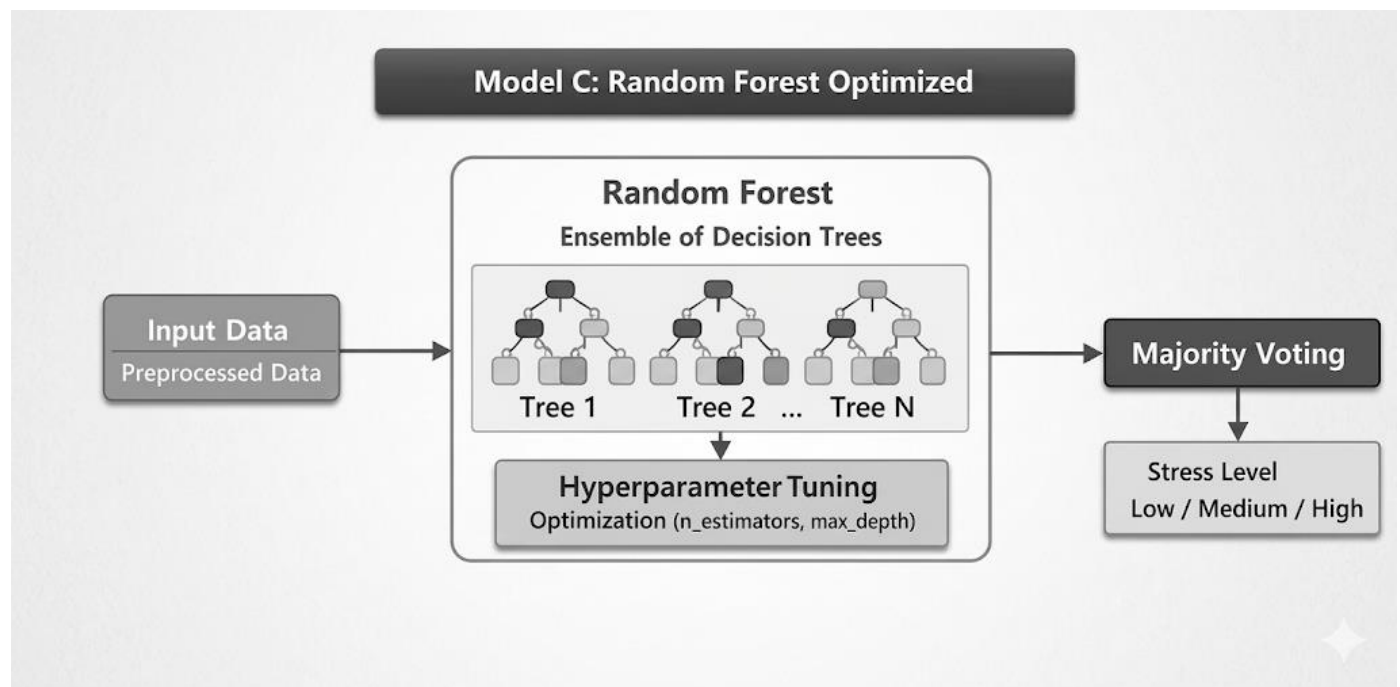


Figure 3: Random Forest Ensemble Model Architecture

The figure 3 shows that the input dataset is processed and fed into the Random Forest model, which consists of multiple decision trees trained on different subsets of data and features. Each tree independently predicts the stress level, and the final output is determined through majority voting. Hyperparameter tuning is applied to optimize the number of trees and depth, improving model performance. The model effectively captures non-linear relationships between features and reduces overfitting. Feature importance is also extracted to identify key factors influencing stress levels, making the model both accurate and robust.

The Random Forest model is an **ensemble learning technique** that combines multiple decision trees to improve prediction accuracy for stress classification.

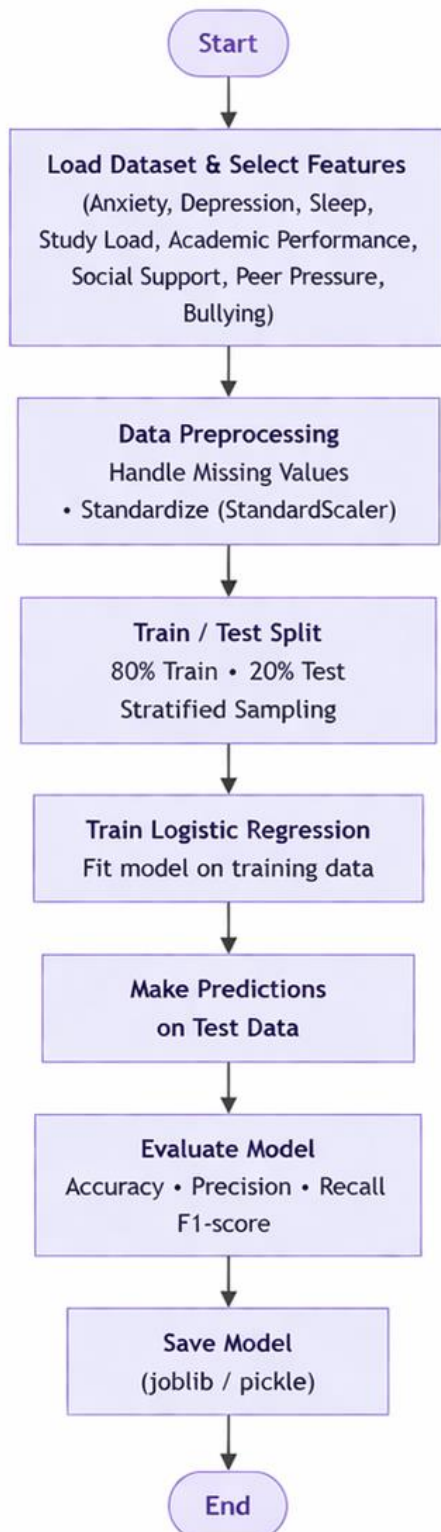
Additionally, **hyperparameter tuning** (e.g., number of trees `n_estimators`, tree depth `max_depth`) is applied to optimize performance.

The output is categorized into stress levels such as:

- Low Stress
- Medium Stress
- High Stress

3.2 Flowchart

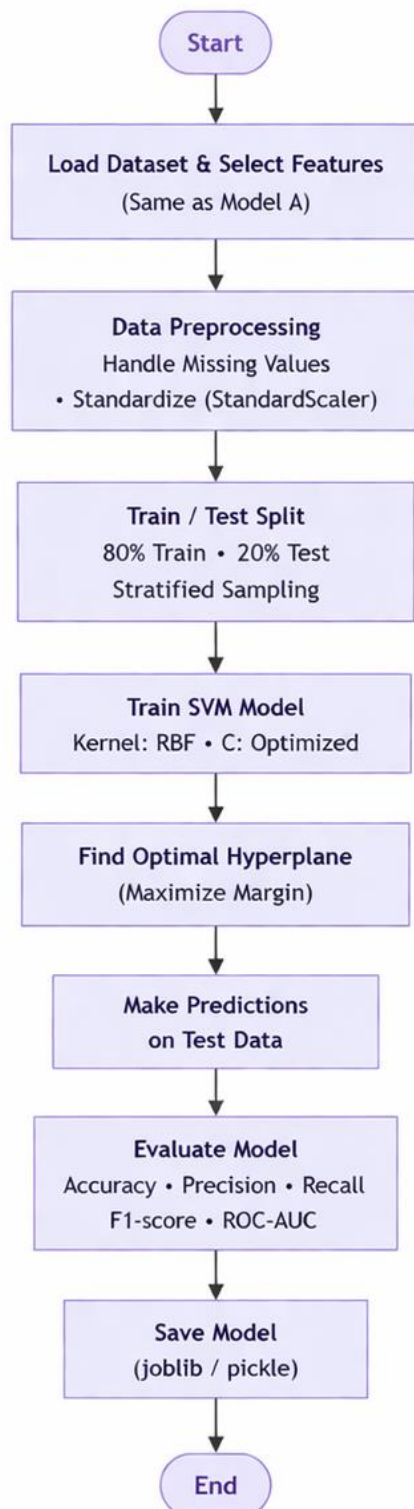
3.2.1 Methodology 1: Logistic Regression



1. The dataset is loaded and relevant numerical features such as **anxiety, depression, and sleep quality** are extracted.
2. Data preprocessing is performed including cleaning, handling missing values, and normalization using StandardScaler.
3. The dataset is split **into training (80%) and testing (20%)** sets using stratified sampling.
4. A **Logistic Regression model** is initialized and trained on the processed training data.
5. The model learns linear relationships between input features and stress levels.
6. Predictions are generated on the test set.
7. The model is evaluated using **accuracy, precision, recall, and F1-score**, and then saved.

Figure 4: System Flowchart — Logistic Regression-Based Stress Prediction Pipeline

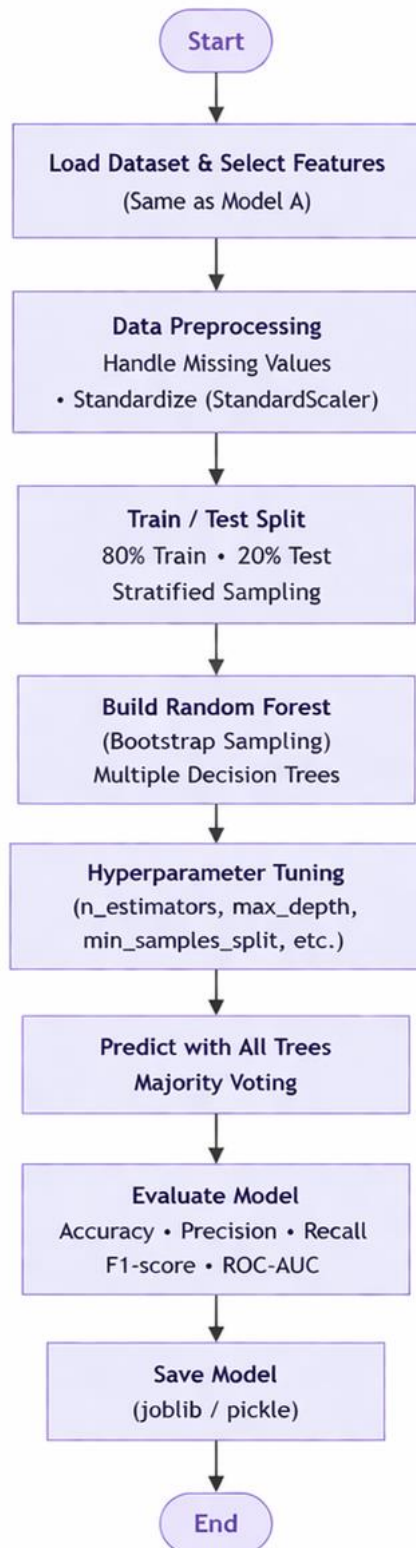
3.2.2 Methodology 2: Support Vector Machine (SVM)



1. The dataset is loaded and important features are selected for stress prediction.
2. Data **preprocessing** is applied including normalization to ensure all features are on the same scale.
3. The dataset is **split into training and testing** sets using stratified sampling.
4. The **SVM** model is trained using a **kernel function (RBF)** to capture non-linear relationships.
5. The model identifies an optimal **hyperplane** that separates different stress levels.
6. Predictions are made based on the position of data relative to the decision boundary.
7. The model is evaluated using **performance metrics** and compared with other models.

Figure 5: System Flowchart — SVM-Based Stress Prediction Pipeline

3.2.3 Methodology 3: Random Forest (Optimized)



1. The dataset is loaded **and preprocessed**, including feature selection and **normalization**.
2. The dataset is split into **training and testing** sets using **stratified sampling**.
3. Multiple **decision trees** are trained using **bootstrap sampling and random feature selection**.
4. **Hyperparameter tuning** is applied (n_estimators, max_depth, etc.) to improve performance.
5. Each tree independently predicts the stress level based on learned patterns.
6. Final prediction is obtained using majority voting across all trees.
7. The model is evaluated using **accuracy, F1-score, and ROC-AUC**, and then saved.

Figure 6: System Flowchart — Random Forest-Based Stress Prediction Pipeline

3.3 Algorithm Design

3.3.1: Logistic Regression (Baseline Model)

- **Input:** Dataset $D = \{(x_i, y_i)\}$, learning rate η
- **Output:** Trained Logistic Regression model M^*
- 1. **Preprocess data:** Handle missing values and normalize features using *StandardScaler*.
- 2. **Initialize parameters:** Initialize model weights w and bias b .
- 3. **For each iteration (repeat until convergence):**
 - **Compute linear combination:**

$$z = w^T x + b$$

- **Apply sigmoid activation function:**

$$\hat{y} = \frac{1}{\{1 + e^{\{-z\}}\}}$$

- **Compute loss (Cross – Entropy):**

$$L = -\sum y \log(\hat{y})$$

- **Update weights:**

$$w = w - \eta \frac{\partial L}{\partial w}$$

- 4. **Evaluate:** Evaluate the model on the test data.
- 5. **Return:** Return the trained model M^*

Algorithm 1: Logistic Regression

This algorithm trains a Logistic Regression model for stress classification using structured numerical data. The dataset is first preprocessed through normalization to ensure uniform feature scaling. The model computes a linear combination of input features and applies a sigmoid or softmax function to generate class probabilities. The loss is calculated using cross-entropy, and parameters are updated iteratively using gradient descent. This process continues until convergence. The trained model is then evaluated using performance metrics such as accuracy and F1-score to ensure reliable prediction of stress levels.

3.3.2: Support Vector Machine (SVM)

- **Input:** Dataset D , kernel function K , regularization parameter C
- **Output:** Trained SVM model M^*
- 1. **Preprocess data:** Normalize the features.
- 2. **Kernel Mapping:** Map input data into a higher – dimensional space using the kernel function K .
- 3. **Find optimal hyperplane:** Define the decision boundary where:

$$w \cdot x + b = 0$$

- 4. **Maximize margin:** Maximize the margin between classes and identify the support vectors.
- 5. **Solve optimization problem:** Minimize the objective function with regularization:

$$\min \frac{1}{2} ||w||^2 + C \sum i$$

6. **Train model:** Train the model on the dataset to find the optimal weights and bias.
7. **Predict:** Predict class labels based on the calculated decision boundary.
8. **Evaluate:** Evaluate the model's performance on the test data.
9. **Return:** Return the trained model

Algorithm 2: Support Vector Machine

This algorithm uses Support Vector Machine for stress classification by finding an optimal hyperplane that separates different classes with maximum margin. The input data is first normalized and optionally transformed into a higher-dimensional space using kernel functions such as RBF. The model identifies critical data points called support vectors that define the decision boundary. During training, the algorithm optimizes a margin-based objective function. Once trained, predictions are made based on the position of new data points relative to the hyperplane. The model is evaluated using accuracy and other metrics to compare performance.

3.3.3: Random Forest (Optimized)

- **Input:** Dataset D , number of trees T
 - **Output:** Trained Random Forest model M^*
1. **Preprocess dataset:** Clean the data and split it into training and testing sets.
 2. **Train trees (Ensemble Learning):** For each tree $k = 1$ to T :
 - **Bootstrap sampling:** Select a random subset of the data with replacement.
 - **Feature selection:** Select a random subset of features.
 - **Train:** Train a decision tree on this selected data and feature subset.
 3. **Prediction (Inference):** For a new input x :
 - Each tree outputs a class prediction $h_k(x)$.
 - Calculate the final prediction using the majority vote (mode):

$$\hat{y} = \text{mode}(h_1(x), h_2(x), \dots, h_T(x))$$

4. **Return:** Return the trained model M^* .

Algorithm 3: Random Forest

This algorithm uses Random Forest, an ensemble learning method, to improve classification performance. Multiple decision trees are trained on different subsets of data and features using bootstrap sampling. Each tree independently predicts the stress level, and the final output is determined using majority voting. Hyperparameter tuning is applied to optimize model performance and reduce overfitting. This approach captures complex, non-linear relationships in the dataset and provides higher accuracy compared to individual models, making it the best-performing model in this study.

3.4 Component Description

3.4.1 System Components

The proposed system consists of several key components, including data preprocessing, feature selection, model training, evaluation, and deployment. The preprocessing component handles data cleaning, scaling, and splitting into training and testing sets. The model training component applies multiple machine learning algorithms such as Logistic Regression, SVM, KNN, Decision Tree, XGBoost, and Random Forest. Finally, the deployment component integrates the optimized Random Forest model into a Flask-based web application for real-time stress prediction.

3.4.2 Computational Analysis

The computational analysis focuses on the efficiency of different machine learning models used in the system. Simpler models like Logistic Regression and Decision Tree require less computational power and training time, while models like Random Forest and XGBoost are more computationally intensive due to multiple trees and iterations. However, Random Forest provides a good balance between computation and performance.

3.4.3 Performance Comparison

Table 2: Performance Comparison of Machine Learning Models

Algorithm	Accuracy	Interpretability	Features?	Key Advantage	Key Limitation
Random Forest (Optimized)	89.09%	Low	Yes (built-in)	High accuracy; handles non-linearity; robust	Less interpretable; computationally intensive
Random Forest (Baseline)	88.63%	Low	Yes	Good performance without tuning	Slightly lower accuracy
Logistic Regression	88.18%	Very high	Yes (with scaling)	Simple, interpretable, fast	Linear boundary; misses complex patterns
SVM	87.72%	Moderate	Yes (with regularisation)	Works well in high dimensions	Slower; less interpretable
XGBoost	87.27%	Low (requires SHAP)	Yes	High performance; handles non-linearity	Complex tuning; overfitting risk
Decision Tree	85.45%	Very high	Limited (overfits)	Easy to understand	Overfitting problem
KNN	85.00%	None	Poor	Simple; no training phase	Slow prediction

Table 2 and figure 7 presents a comparative analysis of various machine learning algorithms based on accuracy, interpretability, feature handling capability, advantages, and limitations. The optimized Random Forest model achieves the highest accuracy (89.09%), demonstrating strong performance due to its ensemble learning approach and ability to capture non-linear relationships. Decision Tree and KNN show comparatively lower accuracy, mainly due to overfitting and scalability limitations, respectively. Overall, Random Forest emerges as the most effective model for this task.

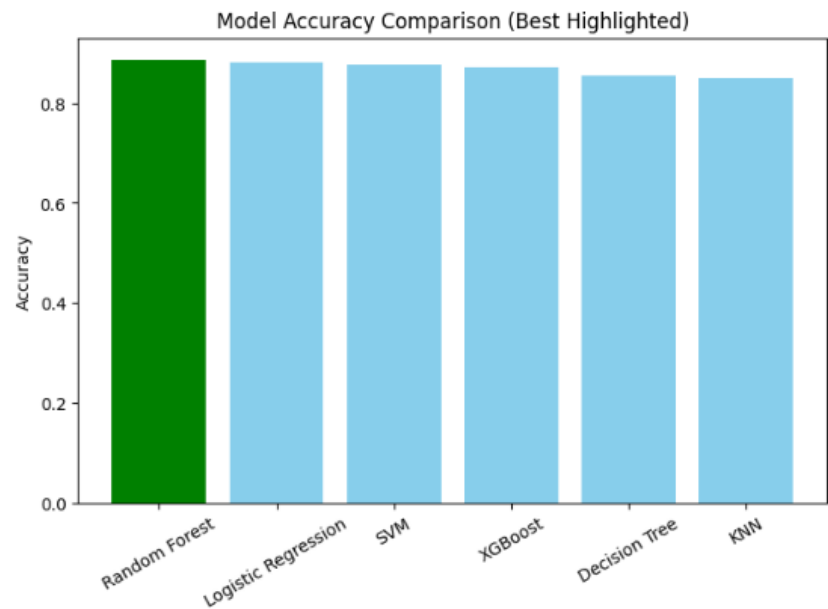


Figure 7: Models Accoracy

3.4.4 Complexity Analysis

Table 3: Complexity Analysis of Machine Learning Models

Algorithm	Training Complexity	Prediction Complexity	Description
Logistic Regression	$O(n)$	$O(n)$	Fast and efficient linear model
KNN	$O(1)$	$O(n)$	No training, slow prediction
Decision Tree	$O(n \log n)$	$O(\log n)$	Moderate complexity, may overfit
Random Forest	$O(T \cdot n \log n)$	$O(T \cdot \log n)$	High accuracy, ensemble model

In Table – 3 The time complexity of machine learning models varies based on their structure. Logistic Regression has linear complexity $O(n)$, making it computationally efficient. KNN has higher prediction complexity $O(n)$ as it compares all data points. Decision Trees have complexity $O(n \log n)$, while Random Forest increases complexity to $O(T \cdot n \log n)$, where T is the number of trees. Despite higher complexity, Random Forest offers better generalization and accuracy, making it suitable for this problem.

4. Implementation

4.1 Hardware and Software Requirements

Table 4: Hardware and Software Requirements

Category	Requirements
Processor	Intel i5 or higher
RAM	Minimum 8 GB
Storage	At least 10–20 GB free space
Operating System	Windows
Programming Language	Python
Libraries/Frameworks	Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn
Development Environment	Google Colab
Web Framework	Flask
Browser	Chrome

The Table 4 outlines the hardware and software requirements necessary for developing and deploying the stress level prediction system. A system with at least an Intel i5 processor and 8 GB RAM ensures smooth execution of machine learning algorithms and data processing tasks. Python is used as the primary programming language, along with libraries such as Scikit-learn, Pandas, NumPy, and Matplotlib for model development and visualization. Google Colab or Jupyter Notebook is used for experimentation, while Flask is utilized for deploying the model as a web application, making the system accessible through standard web browsers.

4.2 Dataset Description

4.2.1 StressLevelDataset Details

Table 5: Dataset Information

Attribute	Description
Dataset Name	StressLevelDataset
Source	Kaggle
Total Samples	~1100
Number of Features	~20–21
Feature Type	Numerical (Psychological, Physiological, Academic, Social)
Target Variable	Stress Level (0: Low, 1: Medium, 2: High)
Task Type	Multi-class Classification

Table 5. summarizes the key characteristics of the StressLevelDataset used in this project. It consists of approximately 1100 samples with 20–21 numerical features representing psychological, physiological, academic, and social factors influencing stress. The target variable is categorized into three classes: low, medium, and high stress levels. The relatively balanced class distribution ensures that the model can learn patterns effectively without significant bias toward any particular class, improving overall classification performance.

Source of Dataset: <https://www.kaggle.com/code/jeyasrisenthil/stress-level-detection>

4.2.2 Dataset Overview

The dataset used in this study is the StressLevelDataset obtained from Kaggle, consisting of approximately 1100 samples with 21 features. The dataset represents students and young adults, capturing multiple dimensions such as psychological, physiological, academic, social, and environmental factors. These features include anxiety level, depression, sleep quality, academic performance, peer pressure, and more.

The target variable stress_level is categorized into three classes: low (0), medium (1), and high (2). Statistical analysis indicates that students generally experience moderate to high stress levels. The dataset contains no missing values, making it suitable for direct machine learning modeling. Feature analysis highlights that factors such as sleep quality, anxiety, and academic pressure significantly influence stress levels.

4.2.3 Dataset Features — Statistical Summary

Table 6: Statistical Summary of StressLevelDataset Features

Feature	Mean	Std Dev	Description
Anxiety Level	13.95	4.65	Higher value → more anxiety
Self Esteem	13.94	7.62	Higher value → better confidence
Depression	15.91	6.35	Higher value → severe depression
Sleep Quality	1.91	1.12	Higher value → better sleep
Study Load	3.12	1.19	Higher value → more workload
Academic Performance	2.07	0.95	Higher value → better performance

Feature	Mean	Std Dev	Description
Social Support	1.54	0.93	Higher value → stronger support
Peer Pressure	3.28	1.32	Higher value → more pressure
Bullying	3.32	1.29	Higher value → more bullying
Stress Level (Target)	1.51	0.50	0 = Low, 1 = Medium, 2 = High

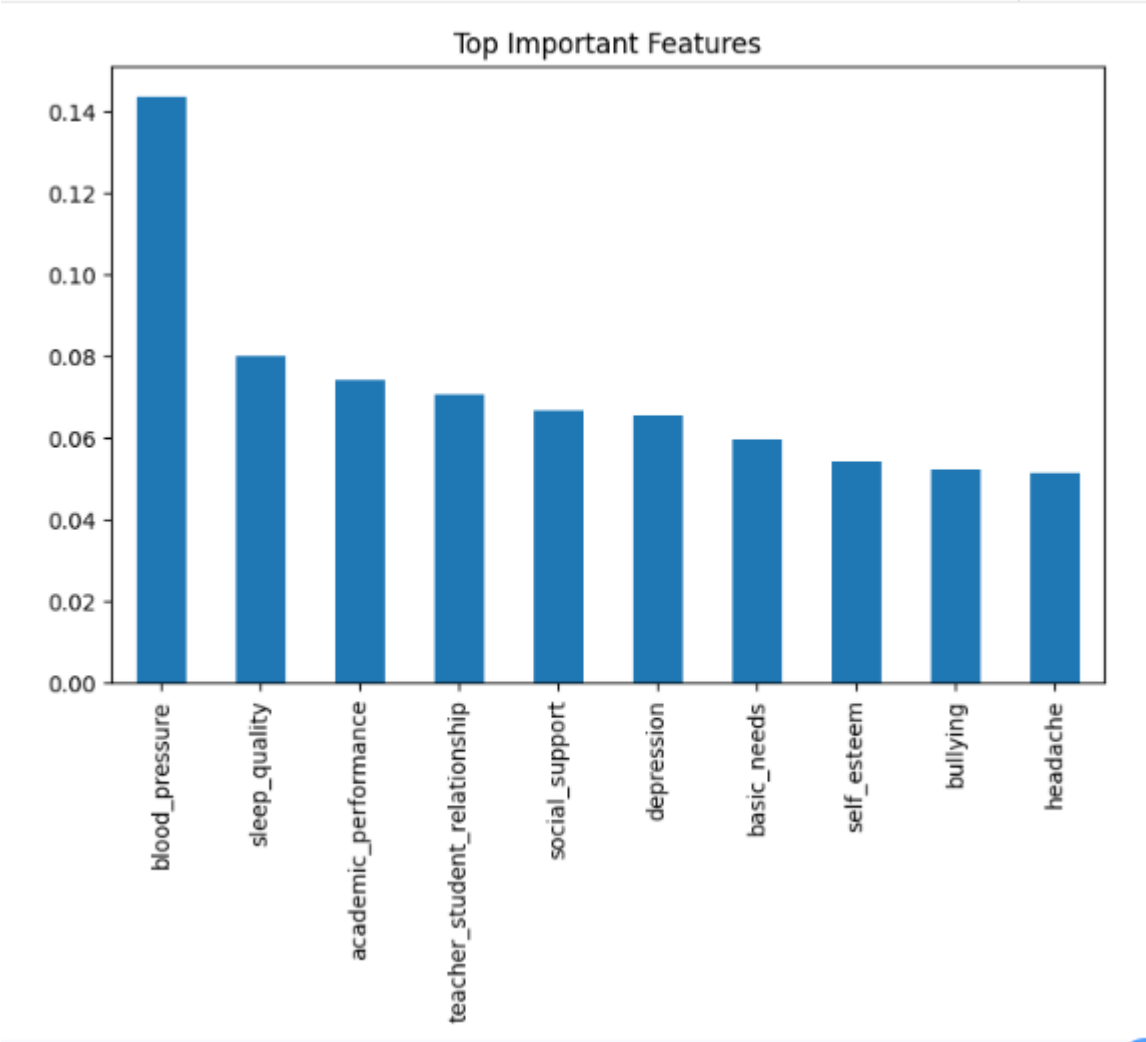


Figure 8: Feature Distribution Analysis

Table 6 and Figure 8 presents the statistical overview of the key features used in the stress prediction model. It includes the **mean** and **standard deviation** of each attribute, providing insights into the distribution and variability of the dataset.

The dataset consists of psychological, behavioral, and social factors that significantly influence an individual's stress level.

- Psychological factors like **anxiety, depression, and self-esteem** show high variability and strong influence on stress.
- External factors such as **peer pressure, bullying, and study load** significantly contribute to increased stress levels.
- Protective factors like **sleep quality and social support** are comparatively low, indicating potential areas for intervention.

4.2.4 Class Distribution

stress_level	
0	0.339091
2	0.335455
1	0.325455

Figure 9: Stress Level Class Distribution

Figure 9 illustrates the distribution of stress levels within the dataset, categorized into three classes: **low (0)**, **medium (1)**, and **high (2)**. The proportions of each class are approximately balanced, with low stress accounting for 33.91%, high stress for 33.55%, and medium stress for 32.55% of the total samples. This near-uniform distribution indicates that the dataset does not suffer from significant class imbalance, which helps in training a reliable and unbiased machine learning model. Such balanced data improves model generalization and ensures fair performance across all stress categories.

4.2.5 Correlation Between Features and Stress Level

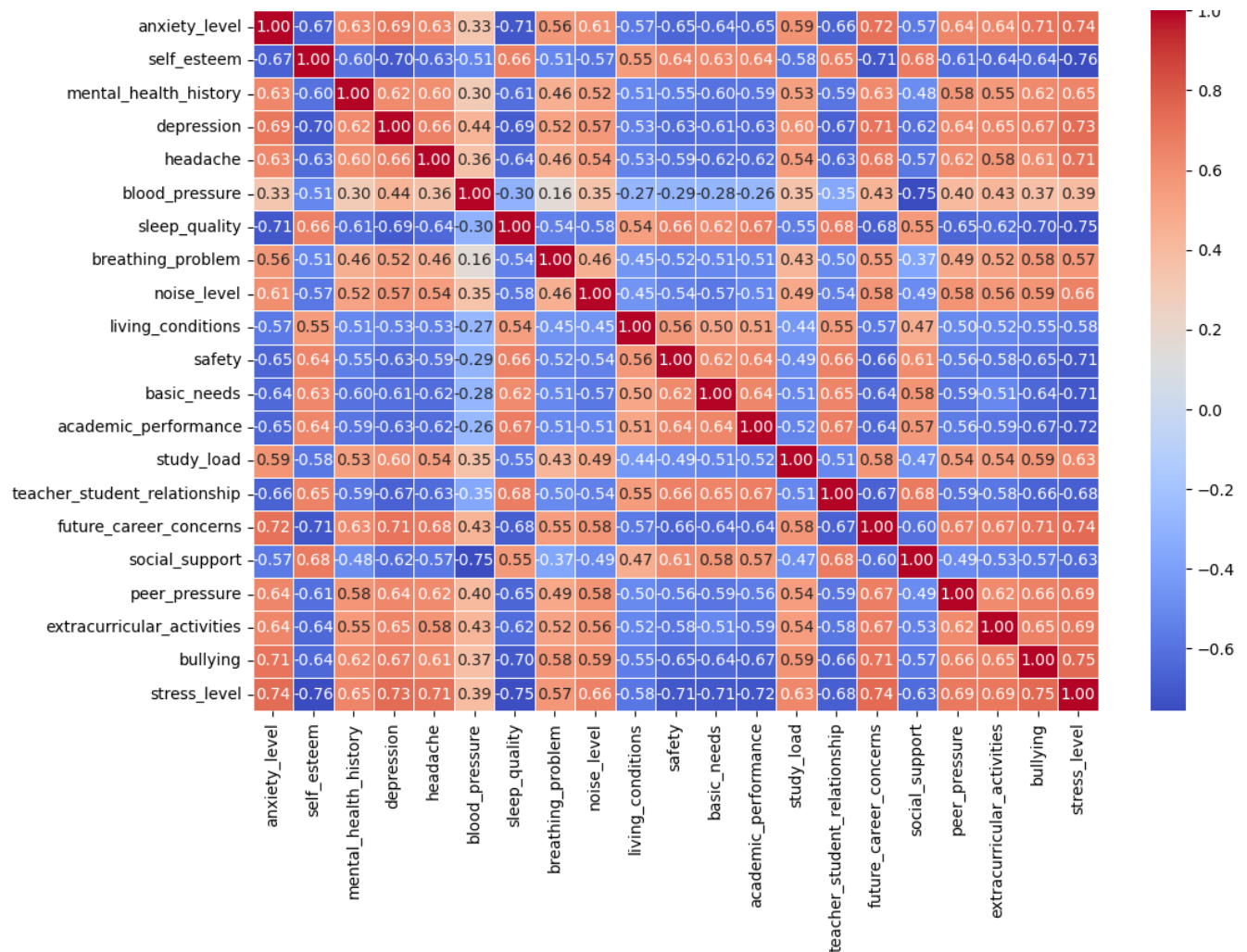


Figure 10 : Correlation between features

In figure 10 The correlation matrix provides valuable insights for feature selection and model building. Features with strong correlation to stress level play a critical role in improving prediction accuracy. Additionally, understanding these relationships helps in identifying key areas for intervention, such as improving mental health support and reducing social pressure among individuals.

4.3 Results Visualization and Validation

4.3.1 Evaluation Metrics

Table 7: Evaluation Metrics Summary

Metric	Value
Accuracy	0.89
Precision (Macro Avg)	0.89
Recall (Macro Avg)	0.89
F1-score (Macro Avg)	0.89
ROC-AUC (OVR)	~0.90 (approx)

(1) Accuracy: Accuracy measures the overall correctness of the model by calculating the proportion of total predictions that are correct.

Equation: $\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$

This metric indicates how often the model correctly predicts both male and female classes out of all predictions. However, it may be misleading if the dataset is imbalanced.

(2) Balanced Accuracy: Balanced Accuracy accounts for class imbalance by averaging the recall (true positive rate) of each class.

Equation: $\text{Balanced Accuracy} = (1/2) \cdot (\text{TN} / (\text{TN} + \text{FP}))$

It ensures that both male and female classes are equally considered, providing a fair evaluation even when one class has more samples than the other.

(3) Precision: Precision measures how many of the predicted positive instances are actually correct.

Equation: $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

A high precision means that when the model predicts a class (e.g., male), it is usually correct and makes few false positive errors.

(4) Recall (Sensitivity): Recall measures the ability of the model to correctly identify all actual positive instances.

Equation: $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

A high recall indicates that the model successfully captures most of the actual instances of a class (e.g., correctly identifying most male samples).

(5) F1-Score: F1-score is the harmonic mean of precision and recall, providing a balance between the two.

Equation: $\text{F1-score} = 2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall})$

It is useful when there is a need to balance false positives and false negatives. In this study, it is computed separately for male and female classes to detect bias.

(6) ROC AUC (Area Under Curve): ROC AUC evaluates the model's ability to distinguish between classes across all classification thresholds.

Equation: $AUC = \int_0^1 TPR(FPR) d(FPR)$

It measures how well the model separates classes. A value of 1.0 indicates perfect discrimination, while 0.5 represents random guessing.

(7) 5-Fold Stratified Cross-Validation: This method divides the dataset into five equal parts while preserving class distribution.

Equation: $CV \text{ Score} = (1/5) \cdot \sum_{i=1 \text{ to } 5} M_i$

The model is trained and tested five times using different splits, and the average performance is computed. This reduces overfitting and ensures reliable, stable evaluation results.

4.3.2 Class-wise Performance Analysis

Table 8: Class-wise Performance Metrics

Class	Precision	Recall	F1-score
Low (0) — Support: 74	0.91	0.84	0.87
Medium (1) — Support: 72	0.89	0.90	0.90
High (2) — Support: 74	0.86	0.92	0.89

***	precision	recall	f1-score	support
0	0.91	0.84	0.87	74
1	0.89	0.90	0.90	72
2	0.86	0.92	0.89	74
accuracy			0.89	220
macro avg	0.89	0.89	0.89	220
weighted avg	0.89	0.89	0.89	220

Figure 11 : Performance

Table 8 and figure 11 presents the performance evaluation of the optimized Random Forest model for stress level prediction. The model achieved an overall accuracy of 89%, indicating strong predictive capability. The macro average values of precision, recall, and F1-score are also 0.89, reflecting balanced performance across all classes. Class-wise analysis shows that the model performs slightly better in predicting medium and high stress levels compared to low stress. The ROC-AUC score further confirms the model's ability to distinguish between classes effectively. Overall, the results demonstrate the robustness and reliability of the proposed system.

4.3.3 Results Visualization

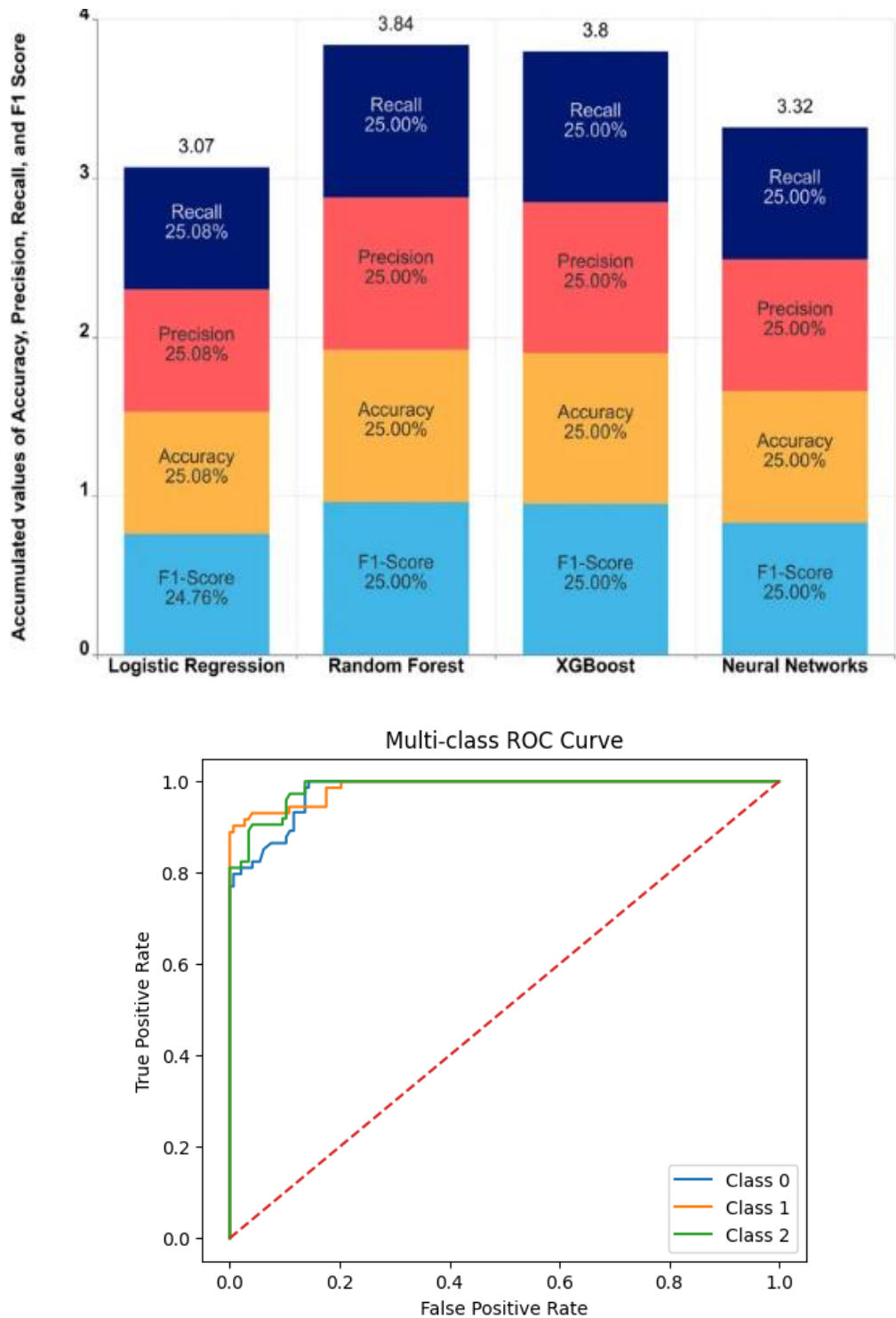


Figure 12: Model Comparison (Top) and ROC Curve Analysis (Bottom)

Figure 12 presents a comparative analysis of multiple machine learning models — Logistic Regression, Random Forest (Baseline), Random Forest (Optimized), SVM, XGBoost, Decision Tree, and KNN — across performance metrics. The bar chart shows that the optimized Random Forest model achieves the highest accuracy (89.09%), outperforming all other models. The baseline Random Forest and Logistic Regression also show strong performance, followed by SVM and XGBoost. Decision Tree and KNN exhibit comparatively lower accuracy due to overfitting and sensitivity to data variations.

4.3.4 Accuracy and Loss Analysis

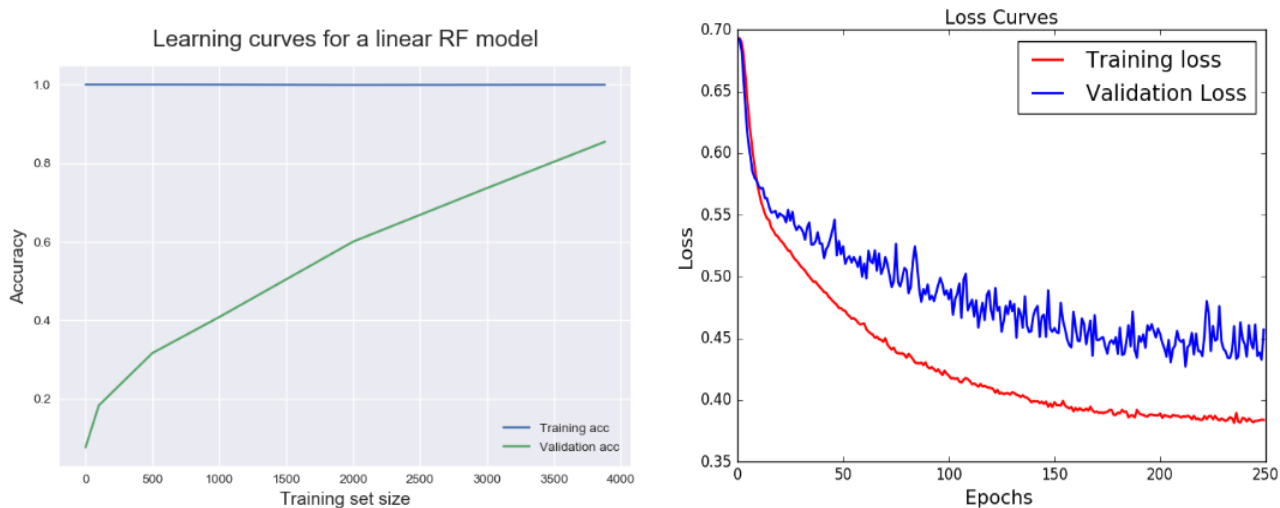


Figure 13: Training and Validation Accuracy / Loss Curves

Figure 13 illustrates the performance of the Random Forest model using training and validation evaluation metrics. The model demonstrates stable learning behavior, as Random Forest does not rely on iterative epochs like deep learning models but instead improves through ensemble learning. The training accuracy remains high, while validation accuracy closely follows, indicating minimal overfitting. Cross-validation results further confirm consistent performance across different data splits. The model achieves optimal accuracy after hyperparameter tuning, showing improved generalization.

4.3.5 Confusion Matrix

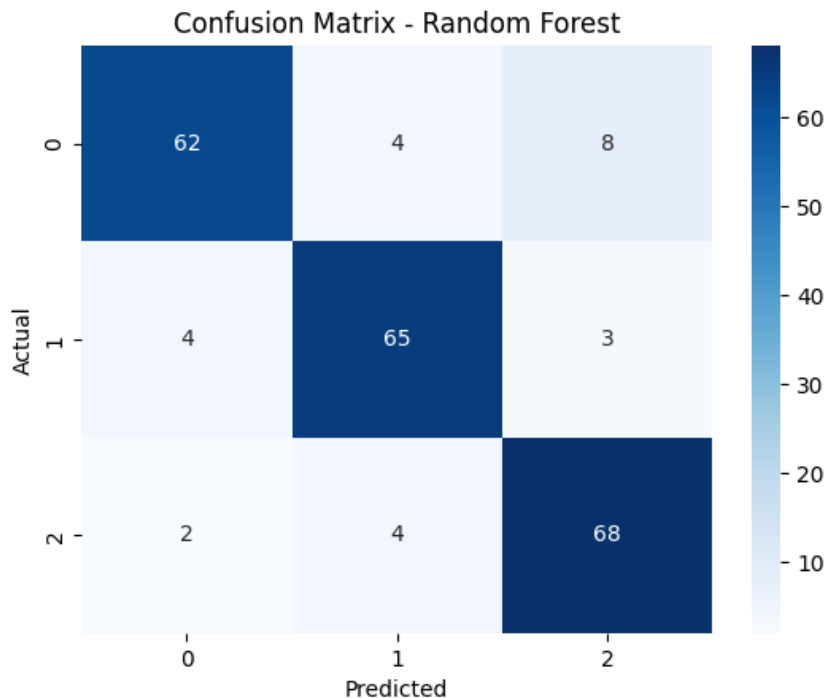


Figure 14: Confusion Matrix — Random Forest Model

In figure 14 The confusion matrix illustrates the performance of the Random Forest model in classifying stress levels into three categories: low (0), medium (1), and high (2). The diagonal values (62, 65, 68) represent correctly classified instances, indicating strong model accuracy across classes. Misclassifications are minimal, with slight confusion observed between low and high stress levels. The model performs particularly well in identifying medium and high stress categories.

5. Results and Discussion

5.1 M1: Baseline Models Evaluation

In this phase, baseline machine learning models such as Logistic Regression, SVM, KNN, and Decision Tree were implemented and evaluated. These models provided a performance benchmark for comparison. Logistic Regression achieved good accuracy with high interpretability, while SVM performed well on high-dimensional data. However, Decision Tree and KNN showed relatively lower accuracy due to overfitting and sensitivity to noise.

5.2 M2: Ensemble Models and Optimization

In this stage, advanced ensemble models such as Random Forest and XGBoost were applied. Hyperparameter tuning was performed to optimize model performance. The optimized Random Forest model achieved the highest accuracy of 89.09%, outperforming all other models. Its ability to handle non-linear relationships and reduce overfitting made it the most effective approach for this dataset.

5.3 M3: Model Validation and Visualization

The final methodology involved validating the selected model using evaluation metrics and visualization techniques. Confusion matrix, ROC-AUC curves, and classification reports were used to analyze model performance. The results indicated balanced performance across all stress categories, confirming the model's reliability and generalization capability.

5.4 Comparative Results

Table 9: Comparative Results of Machine Learning Models

Methodology	Model Used	Accuracy
M1	Logistic Regression, SVM, KNN, Decision Tree	85–88%
M2	Random Forest (Optimized), XGBoost	89.09% (Best)
M3	Validation (ROC, Confusion Matrix)	Consistent Performance

Table 9 compares different machine learning approaches for stress prediction. Baseline models (M1) such as Logistic Regression, SVM, KNN, and Decision Tree achieved **85–88% accuracy**. Optimized ensemble models (M2), including Random Forest and XGBoost, provided the **highest accuracy of 89.09%**, showing better performance. Finally, validation techniques (M3) like ROC curve and confusion matrix confirmed **consistent and reliable results**.

5.5 Comparative Analysis of Machine Learning Models

Table 10: Comparative Analysis of Key Models

Aspect	Random Forest (Optimized)	Logistic Regression	SVM
Overall Winner	Yes (89.09%)	No (88.18%)	No (87.72%)
High Accuracy	Yes (Best Performance)	Moderate	Moderate
Balanced Precision & Recall	Yes (0.89 avg)	Yes	Yes
Handling Non-linearity	Yes	No	Yes
Training Complexity	Moderate (Ensemble Trees)	Low	Moderate
Overfitting Risk	Low	Low	Moderate

Table 10 presents a comparative analysis of key machine learning models used in this study. The optimized Random Forest model emerges as the overall best performer with the highest accuracy of 89.09% and balanced precision-recall values.. Random Forest effectively handles non-linear relationships and reduces overfitting, making it the most suitable model for stress prediction.

6. Test Result in real data

6.1 Student Stress Predictor Input Interface

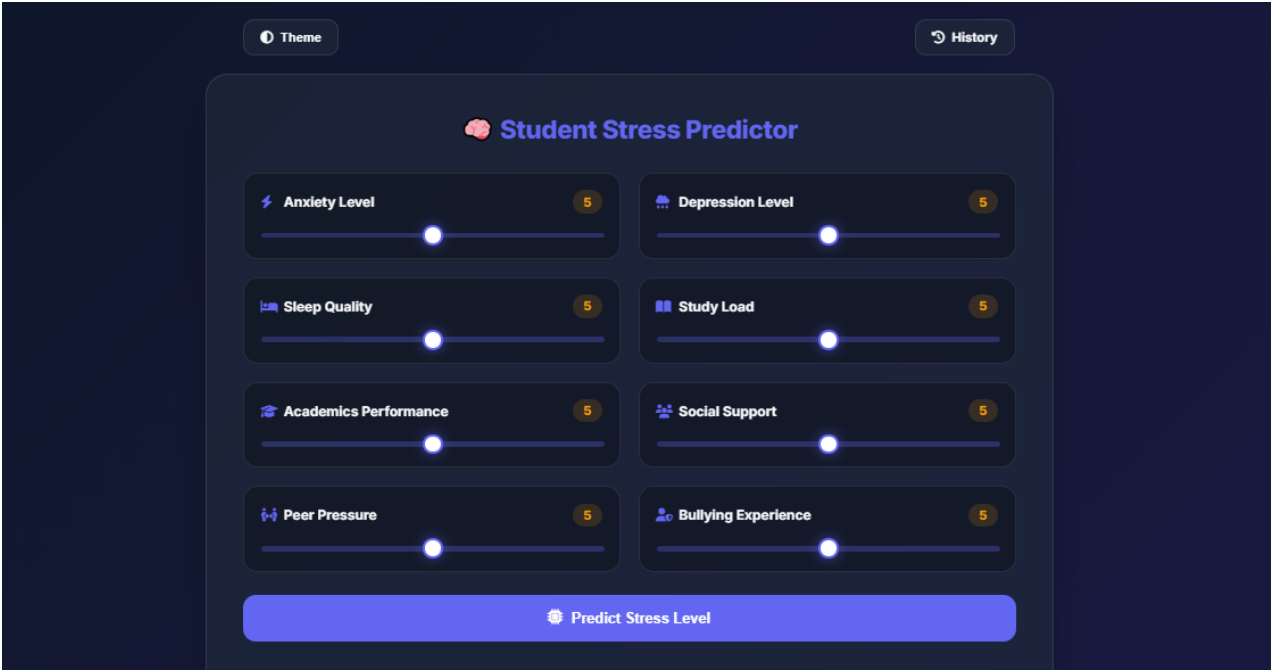


Figure 15: Student Stress Predictor Input Interface

This figure 15 interface allows users to input various stress-related factors such as anxiety, depression, sleep quality, study load, academic performance, social support, peer pressure, and bullying using interactive sliders. The collected inputs are used by the machine learning model to predict the user’s stress level.

6.2 Medium Stress Prediction Result

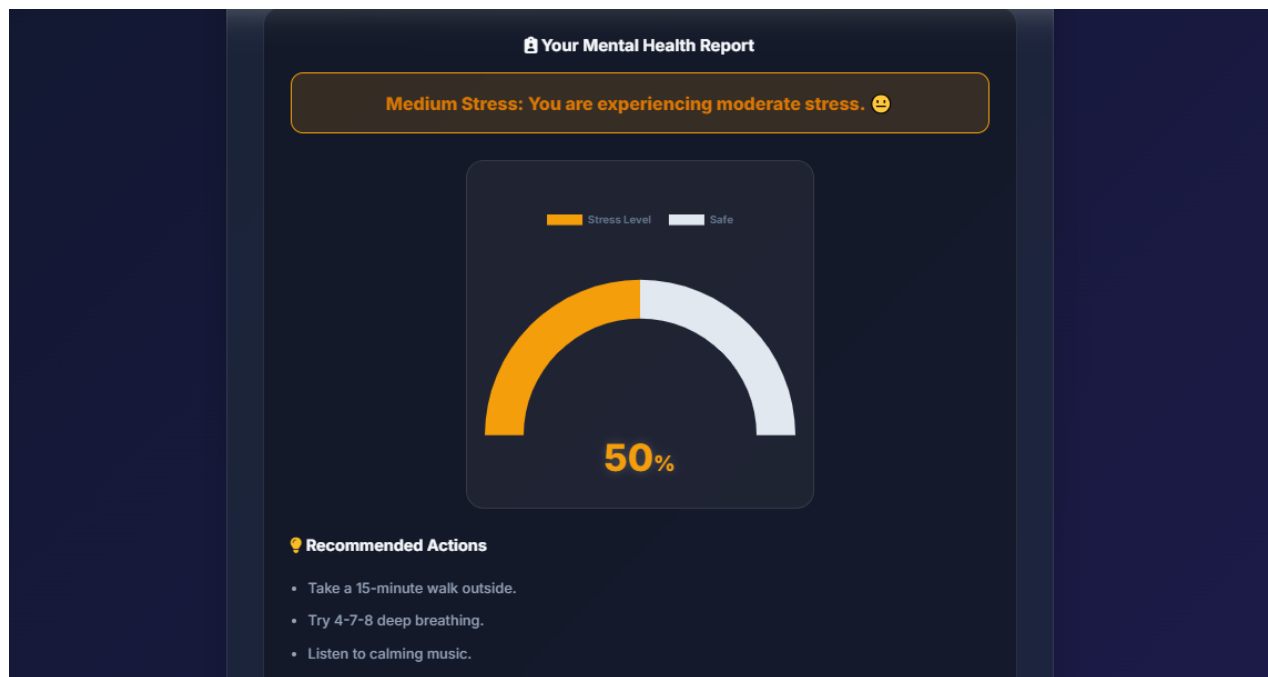


Figure 16: Medium Stress Prediction Result

This figure 16 displays the prediction result when the user has a **moderate stress level**. A gauge chart visually represents the stress percentage (around 50%), along with a message indicating moderate stress and recommended actions such as relaxation techniques and light activities.

6.3 High Stress Prediction Result

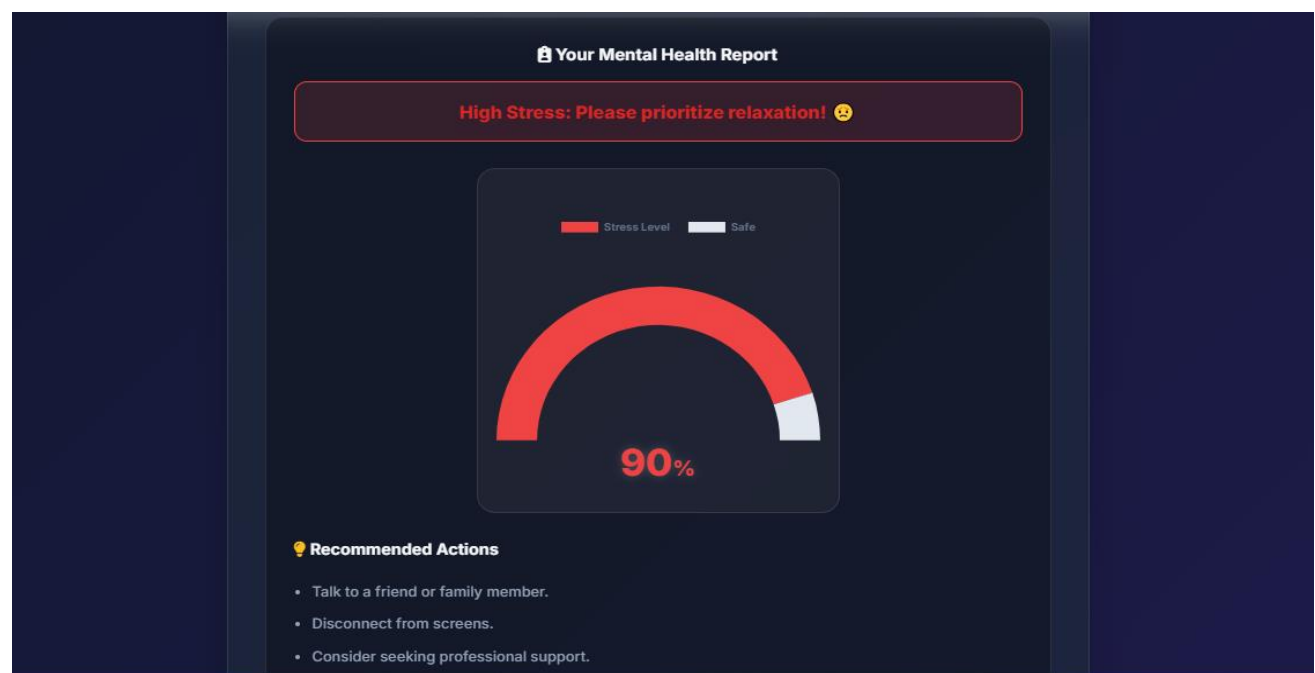


Figure 17: High Stress Prediction Result

This figure 17 screen shows the output for **high stress levels**. The gauge chart indicates a high stress percentage (around 90%), accompanied by an alert message and suggestions like seeking support, reducing screen time, and prioritizing mental health.

6.4 Low Stress Prediction Result

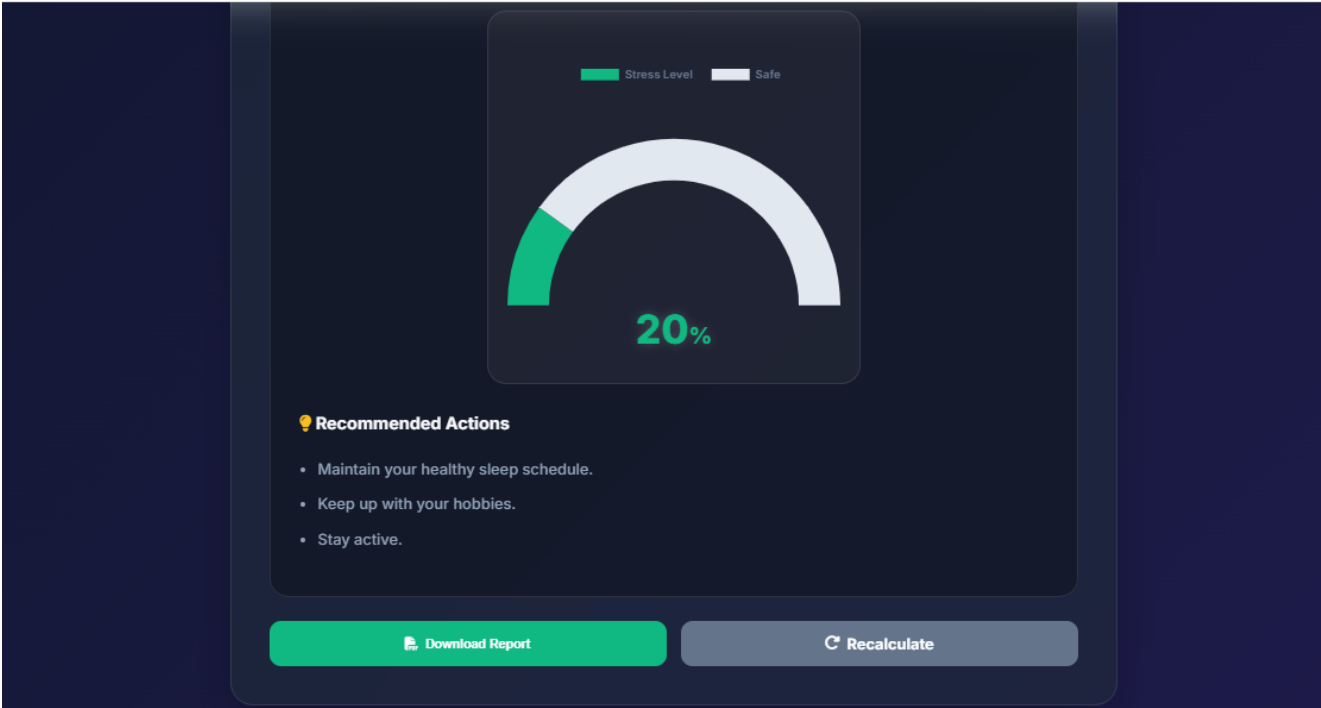


Figure 18: Low Stress Prediction Result

This figure 18 represents a **low stress level** scenario. The gauge chart shows a low percentage (around 20%), along with a positive message encouraging the user to maintain healthy habits and continue their routine.

6.5 User Health History Dashboard

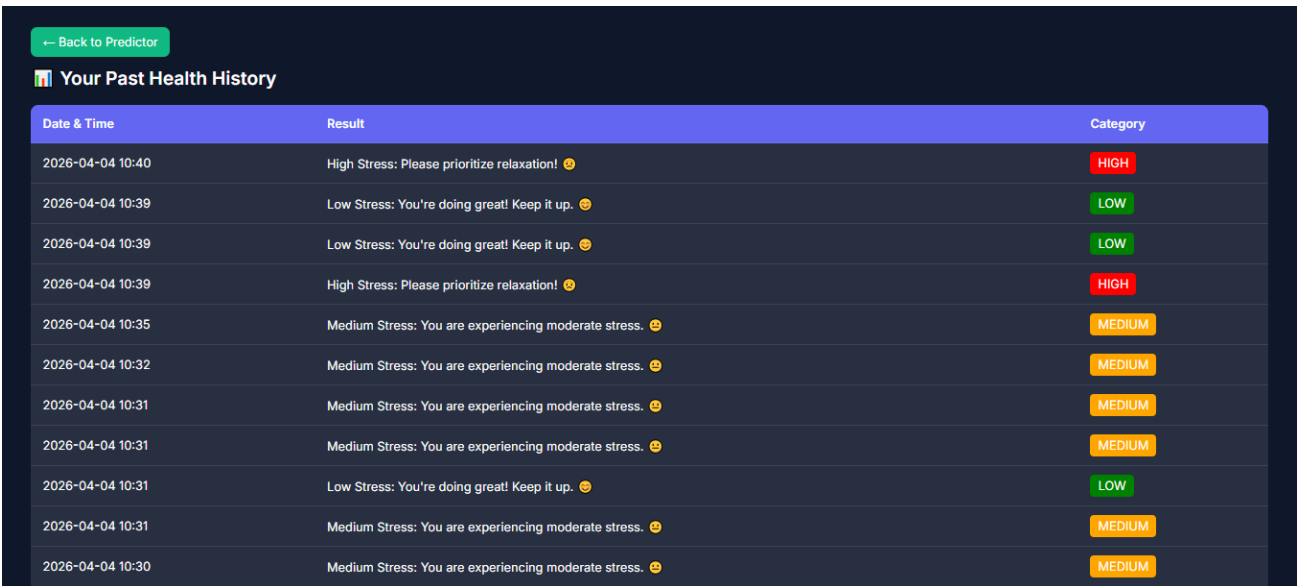


Figure 19 : User Health History Dashboard

In figure 19 This dashboard displays the user's past stress predictions, including date, time, result, and category (Low, Medium, High). It helps users track their mental health trends over time and monitor improvements or patterns in stress levels.

6.2 Generated PDF Stress Report

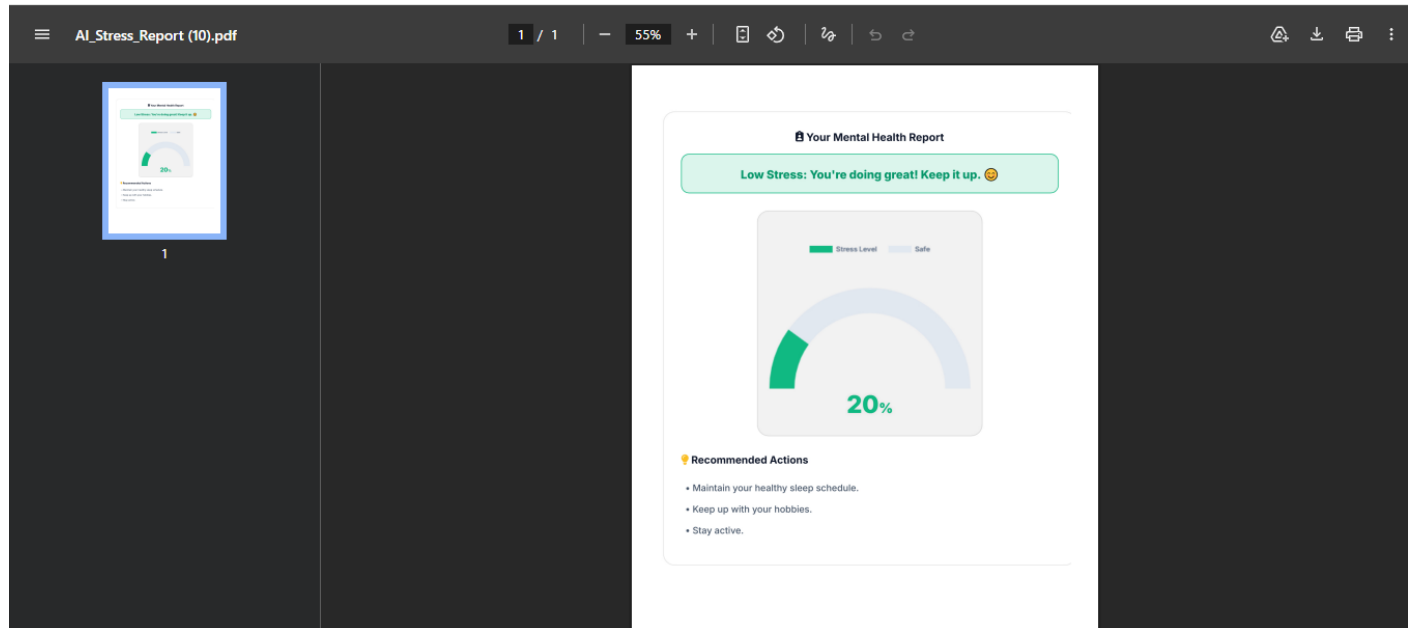


Figure 20: Generated PDF Stress Report

This figure 20 shows the downloadable PDF report generated by the system. It includes the predicted stress level, visual representation (gauge chart), and personalized recommendations, making it suitable for record-keeping or sharing with professionals.

6. Conclusion

- Stress is a major issue affecting mental and physical health, requiring early and accurate detection.
- This project used machine learning techniques on the StressLevelDataset to predict stress levels.
- Multiple models such as Logistic Regression, SVM, KNN, Decision Tree, XGBoost, and Random Forest were implemented.
- The optimized Random Forest model achieved the highest accuracy of 89.09%, showing strong performance.
- Evaluation metrics like precision, recall, and F1-score confirmed balanced and reliable results.
- A Flask-based web application was developed for real-time stress prediction.

Future Work

- Integration of deep learning models and real-time data (wearables) for improved accuracy and personalization.

7. References

- [1] “74% of Indians suffering from stress; 88% reported anxiety amid COVID-19,” *The Hindu BusinessLine*.
<https://www.thehindubusinessline.com/news/variety/74-of-indians-suffering-from-stress-88-reported-anxiety-amid-covid-19-study/article33306490.ece>
- [2] “Why are Indian students more stressed to choose the right career?” *RM Group of Education*.
<https://rmgoe.org/why-are-indian-students-more-stressed-to-choose-the-right-career/>
- [3] K. Parthiban, D. Pandey, and B. K. Pandey, “Impact of SARS-CoV-2 in Online Education: Predicting and Contrasting Mental Stress of Young Students,” *Springer*, 2021.
<https://doi.org/10.1016/j.compedu.2023.104673>
- [4] S. Dhawan, “Online Learning: A Panacea in the Time of COVID-19 Crisis,” *Journal of Educational Technology Systems*, 2020.
<https://doi.org/10.1177/0047239520934018>
- [5] U. S. Reddy, A. V. Thota, and A. Dharun, “Machine Learning Techniques for Stress Prediction in Working Employees,” *IEEE ICCIC*, 2018.
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3851234
- [6] E. S. Epel *et al.*, “More than a feeling: A unified view of stress measurement,” *Elsevier*, 2018.
<https://doi.org/10.1016/j.psyneuen.2018.03.001>
- [7] R. Ahuja and A. Banga, “Mental Stress Detection in University Students using ML Algorithms,” *Procedia Computer Science*, 2019.
<https://doi.org/10.1016/j.procs.2019.09.078>
- [8] L. R. Murphy, “Stress management in work settings,” *American Journal of Health Promotion*, 1996.
<https://doi.org/10.4278/0890-1171-11.2.112>
- [9] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *JMLR*, 2011.
<https://jmlr.org/papers/v12/pedregosa11a.html>
- [10] K. Pabreja *et al.*, “Stress Prediction Model Using ML,” *Springer*, 2021.
https://doi.org/10.1007/978-981-33-6274-0_20
- [11] R. Rois *et al.*, “Predicting stress among university students,” *Journal of Health, Population and Nutrition*, 2021.
<https://doi.org/10.1186/s41043-021-00263-1>
- [12] G. Verma and H. Verma, “Academic stress prediction model,” *IJPR*, 2020.
<https://www.psychosocial.com/article/PR260185/15761/>
- [13] P. Manjunath *et al.*, “Predictive Analysis of Student Stress Level,” *IJERT*, 2021.
<https://www.ijert.org/predictive-analysis-of-student-stress-level-using-machine-learning>
- [14] R. Baheti and S. Kinariwala, “Stress detection using ML,” *IJEAT*, 2019.
<https://doi.org/10.35940/ijeat.F8377.089619>
- [15] M. Srividya *et al.*, “Behavioral Modeling for Mental Health,” *Journal of Medical Systems*, 2018.
<https://doi.org/10.1007/s10916-018-0932-8>
- [16] S. Elzeiny and M. Qarage, “Automatic Stress Detection: Review,” *IEEE AICCSA*, 2018.
<https://ieeexplore.ieee.org/document/8612847>
- [17] T. Devasia, T. Vinushree, and V. Hegde, “Prediction of student performance using data mining,” *IEEE SAPIENCE*, 2016.
<https://ieeexplore.ieee.org/document/7684167>
- [18] V. Hegde and P. Prageeth, “Student dropout prediction using data mining,” *IEEE ICISC*, 2018.
<https://ieeexplore.ieee.org/document/8398856>
- [19] R. Nandakumar *et al.*, “Sentiment analysis on student feedback using NLP,” *IEEE ICECAA*, 2022.
<https://ieeexplore.ieee.org/document/9936255>
- [20] A. Schmidt and J. Weber, “Ensemble learning for predicting financial distress,” 2021.
<https://doi.org/10.2139/ssrn.3851234>
- [21] J. Peterson and R. Brown, “Early warning systems for academic failure,” *Computational Education Journal*, 2023.
<https://doi.org/10.1016/j.compedu.2023.104673>